UNIDAD 5

CONVERSIÓN Y ADAPTACIÓN DE DOCUMENTOS XML

MÓDULO: Lenguaje de marcas y sistemas de gestión

Profesor: José Repiso Rodríguez

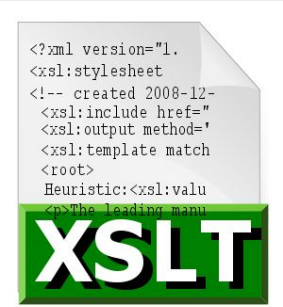
1º DAM/DAW

RA 5: Realiza conversiones sobre documentos para el intercambio de información utilizando técnicas, lenguajes y herramientas de procesamiento.

Criterios de evaluación:

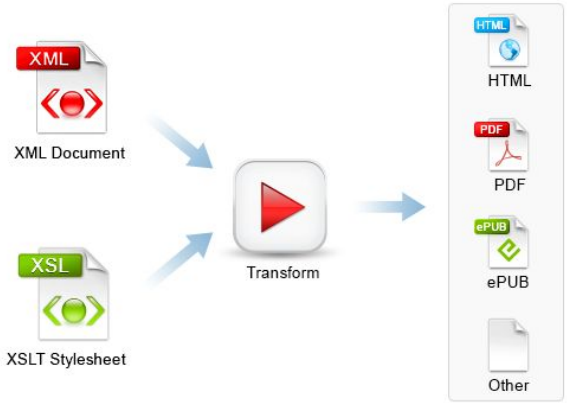
1. Se ha identificado la necesidad de la conversión de documentos para el intercambio de la información.
2. Se han establecido ámbitos de aplicación.
3. Se han analizado las tecnologías implicadas y su modo de funcionamiento.
4. Se ha descrito la sintaxis específica utilizada en la conversión y adaptación de documentos para el intercambio de información.
5. Se han creado especificaciones de conversión.
6. Se han identificado y caracterizado herramientas específicas relacionadas con la conversión de documentos para el intercambio de información.
7. Se han realizado conversiones sobre documentos para el intercambio de información.

Índice



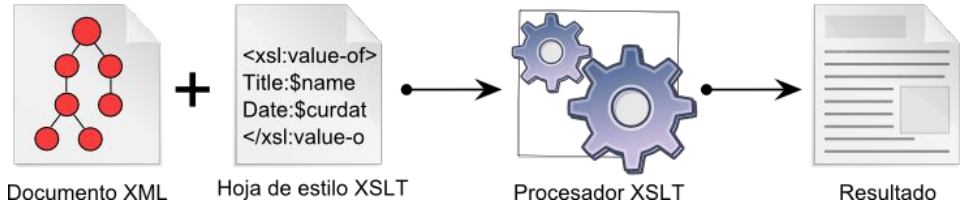
1. Los lenguajes XSL
2. XPath
   a. Tipos de nodos
   b. Expresiones
3. XSLT
   a. Estructura
   b. Transformación
   c. Elementos de XSLT
4. XSL-FO
   a. Instalación de Apache FOP
   b. Estructura
   c. Sintaxis
   d. Conversión

1. Los lenguajes XSL



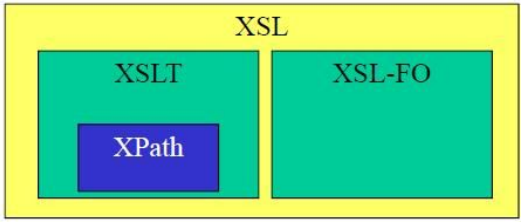
En el ecosistema de XML existen herramientas que permiten **transformar** los documentos, con el fin de adaptar su **representación** o **presentación** al dispositivo o aplicación de salida, es decir, modificar los documentos XML para poder ser mostrados al usuario.

Además, tenemos la opción de realizar adaptaciones incluso a **otros lenguajes de marcas**. Entre sus aplicaciones, podemos realizar transformaciones de los documentos **XML** a **HTML**, a **texto plano**, a formato **CSV**, a sentencias **SQL**, a **ePUB**, a **imágenes**, a documentos **PDF**, a objetos **JSON**, o a cualquiera de los dialectos derivados de XML.



1. Los lenguajes XSL

La familia de lenguajes **XSL (eXtensible Stylesheet Language)** es un conjunto de tecnologías, desarrolladas por el W3C, con el objetivo de proporcionar herramientas para la **transformación de documentos XML**. Esta familia se compone de tres lenguajes:



XPath

XML Path Language se usa recorrer un documento XML utilizando expresiones, permitiendo **seleccionar** y **diferenciar partes** del documento. Es el análogo al DOM en HTML.

XSLT

XSL Transformation es el lenguaje de **transformación** propiamente dicho, el que se encarga de **cambiar la estructura o sintaxis** del documento. Normalmente se usa en conjunto con *XPath*.

XSL-FO

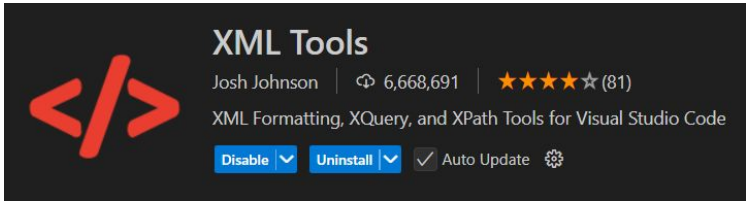
XSL Formatting Objects es el lenguaje que indica el **formato** que debe tener el documento una vez transformado, para poder ser **representado** en diferentes dispositivos o medios.

2. XPath

<xpath>

El primer lenguaje de la familia **XSL** que vamos a ver es **XPath**, ya que es necesario para aplicarlo en **XSLT**. Su finalidad principal es **seleccionar nodos** o conjuntos de nodos de un documento XML. El lenguaje de XPath no tiene una sintaxis derivada de XML, sino que se basa en **expresiones**.
Estas expresiones están basadas en las **rutas** (*path expression*), y se construyen a partir de palabras clave, símbolos y operandos.

Para ejecutar XPath es recomendable usar la extensión **XML Tools** de Visual Studio. Una vez instalado, se puede abrir la paleta de comandos con **Ctrl+Shift+P**, y buscar los comandos:



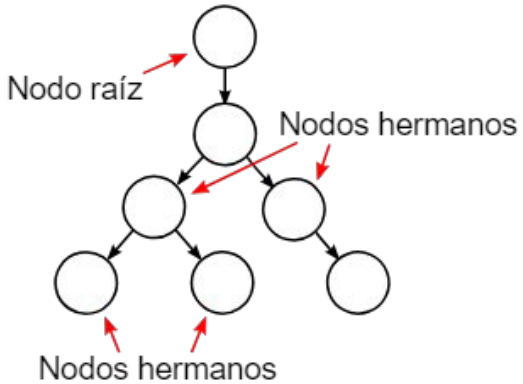
XML Tools: Evaluate XPath — Permite introducir una **expresión** y buscar sus resultados en el documento XML

XML Tools: Get current XPath — Devuelve la **expresión** que permite acceder al nodo seleccionado del documento XML

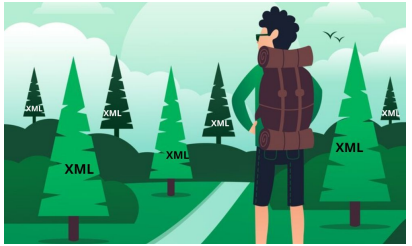
2.1. Tipos de nodos

XPath maneja los documentos como estructura en forma de **árbol**. Dicho árbol está compuesto por **nodos**. Se pueden distinguir los siguientes tipos de nodos:

Document	Representa el documento XML completo
Element	Representa los elementos de XML
Attribute	Representa los atributos de un elemento de XML
Text	Encapsula el contenido de texto interno de un elemento de XML
Namespace	Representa el enlace de un URI de un espacio de nombres o su prefijo



2.2. Expresiones



Las expresiones de ruta de XPath se componen de tres partes:

- **axis** — Indica la **dirección** de búsqueda dentro del documento. Es **opcional**.
- **node_test** — Define qué **nodo** o tipo de nodo queremos buscar.
- **predicate** — **Filtra** los resultados de búsqueda con más condiciones. Es **opcional**.

La estructura es, por tanto: `/axis::node_test[predicate]`

Las **barras /** funcionan de forma similar a las rutas: sirven para indicar el camino que hay que recorrer para realizar la búsqueda, **comenzando desde el elemento raíz**. En XPath, tenemos dos opciones:

- **Barra simple (/):** busca nodos que sean **hijos directos**.
- **Barra doble (//):** busca cualquier **descendiente**, ya sea hijo directo o no.

2.2. Expresiones

Ejemplo_7_1

Node_test

Sirve para seleccionar el nodo como tal. Lo más sencillo es usar el **nombre de las etiquetas** para seleccionar nodos. Si queremos seleccionar todos los nodos posibles, usamos *.
Con esto, ya podemos comenzar a seleccionar nodos en XPath. Por ejemplo:

/obras	Selecciona los nodos ***obras***, hijos del elemento raíz, incluyendo todo su contenido interno (también de sus descendientes)
/obra	Selecciona los nodos ***obra***, hijos directos del elemento raíz. En este caso, por tanto, no devolverá nada, ya que el elemento ***obra*** no es el elemento raíz
/obras/museo	Selecciona los nodos ***museo***, hijos directos de un elemento ***obras***. En este caso, por tanto, no devolverá nada, ya que ***museo*** no es hijo directo de ***obras***
/obras//museo	Selecciona los nodos ***museo***, descendientes de un elemento ***obras***. En este caso, sí devolverá los elementos
/obras/obra/*	Selecciona **todos los nodos** que sean hijos directos de un elemento ***obra***, a su vez hijo directo del elemento ***obras***

2.2. Expresiones

Ejemplo_7_2

Por otro lado, se pueden seleccionar atributos, simplemente precediéndolo por @:

`//obra/@*` ⇨ Selecciona **todos los atributos** de los elementos *obra*, descendientes del elemento raíz

`//obra/@pais` ⇨ Selecciona el atributo **país** de los elementos *obra*, descendientes del elemento raíz

También se pueden indicar **funciones** para una selección más precisa del contenido que se pretende obtener. Dependiendo de su uso, se pueden indicar directamente en la expresión de ruta, o dentro del predicado (que veremos a continuación). Algunas de ellas son:

/text() ⇨ Devuelve solo el contenido de texto de un nodo

string(*nodo*) ⇨ Devuelve todo el contenido de un nodo, pero en formato de cadena de texto

substring(*nodo, p1, p2*) ⇨ Devuelve el contenido entre las posiciones *p1* y *p2* indicadas

contains(*nodo,"texto"*) ⇨ Indica si el nodo contiene o no el *texto* indicado

2.2. Expresiones

Las funciones marcadas en **rojo** solo funcionan con **XPath 2.0** (no aplicable en Visual Studio).

position() → Devuelve la posición del nodo actual dentro de la ruta indicada

last() → Devuelve la última posición de los nodos indicados

string-length(*nodo*) → Devuelve la longitud de caracteres del texto del nodo indicado

count(*expresión*) → Devuelve el número de elementos para la expresión indicada

avg(*nodos*) → Devuelve el valor medio de todos los nodos indicados (cuando contienen datos numéricos)

sum(*nodos*) → Devuelve la suma acumulada de todos los nodos indicados (cuando contienen datos numéricos)

max(*nodos*)
min(nodos) → Devuelve el valor máximo / mínimo de todos los nodos indicados (cuando contienen datos numéricos)

starts-with(*nodo*,"*texto*")
ends-with(*nodo*,"*texto*") → Indica si el nodo comienza / termina por el *texto* indicado

upper-case(*nodo*)
lower-case(*nodo*) → Transforma el nodo seleccionado a mayúsculas / minúsculas

La mayoría de estas funciones no se usan por sí mismas, sino dentro de un **predicado**. En ocasiones, también es necesario que se indiquen sobre una expresión de ruta simple, no con rutas anidadas.

2.2. Expresiones

Ejemplo_7_3

Predicate

El **predicado** son reglas adicionales que se indican para **filtrar** los resultados. Se indican entre corchetes **[]**, y pueden incluir distintas expresiones con distintos operadores:

Posición

Indicando **directamente un número** entre [], podemos seleccionar un elemento concreto de los que cumplen la ruta indicada (comenzando por 1). *Ejemplo:* **//obra[1]**

Aritméticos

Permite realizar operaciones matemáticas sobre el contenido seleccionado, como suma (**+**), resta (-), multiplicación (*), división (**div**), división entera (**idiv**), módulo o resto (**mod**). *Ejemplo:* **//obra[1]/@año + 100**

Lógicos

Aplican comparaciones lógicas típicas, como pueden ser igual (**=**), distinto (**!=**), mayor (**>**), menor(**<**), mayor o igual (**>=**), menor o igual (**<=**), o los operadores lógicos (**and**, **or**, **not**). *Ejemplo:* **//obra[@año <= 1876]**

Funciones

Como vimos anteriormente, muchas de las funciones se deben usar dentro de un predicado. *Ejemplo:* **//titulo[starts-with(text(), "S")]**

Todas estas expresiones se pueden combinar para filtrados más precisos.

2.2. Expresiones

Ejemplo_7_4

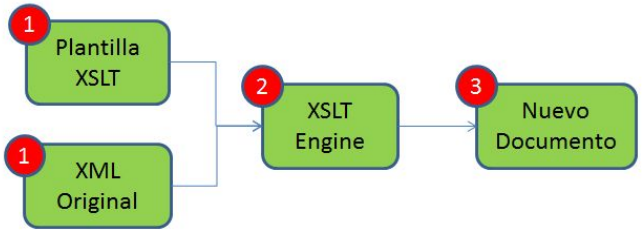
Axis

Los *axis* o ejes permiten "navegar" en una dirección concreta del documento, a partir de una posición relativa. Se indican precedidos de "**::**". Algunos de los más utilizados son:

- **child** → Busca nodos **hijos** del actual (igual que **/**)
- **parent** → Selecciona el nodo **padre** del actual (igual que **.**)
- **attribute** → Selecciona los **atributos** del nodo (igual que **@**)
- **self** → Selecciona el propio nodo **actual** (igual que **.**)
- **descendant** → Busca cualquier nodo **descendiente**, ya sea hijo directo o no (igual que **//**)
- **ancestor** → Busca cualquier nodo **ancestro**, ya sea padre directo o no
- **following-sibling** → Selecciona el **siguiente** nodo **hermano** del actual
- **preceding-sibling** → Selecciona el **anterior** nodo **hermano** del actual

3. XSLT

El segundo de los lenguajes que componen XSL que vamos a estudiar es **XSLT**. Gracias a XSLT podemos **transformar** un documento expresado en **formato XML**, en otro documento en un **formato distinto** (normalmente HTML).



3.1. Estructura básica

Los componentes necesarios para realizar una transformación XSLT son los siguientes:

Archivo XML de entrada

Es el XML original, que se pretende transformar en otro nuevo. Para que sea automático, se puede incluir opcionalmente la **referencia al archivo de transformación**:

```
<?xml-stylesheet type="text/xsl" href="archivo.xsl" ?>
```

Fichero de transformación XSL

Se trata de un archivo con extensión **.xsl**, que contiene las instrucciones para transformar el documento.

Procesador

Es el programa que aplica la transformación.

3.2. Transformación

El archivo de transformación **XSL** también está escrito en formato XML, con la siguiente **estructura general**:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:output method="texto plano o html" encoding="UTF-8"/>
<xsl:template match="/">
        ...Plantilla de transformación en formato XSL...
</xsl:template>
</xsl:stylesheet>
```

⇨ El elemento **xsl:output** sirve para indicar el tipo de archivo de salida que queremos generar. Esto se hace mediante el atributo **method**, que puede tomar los valores *html*, *xhtml*, *xml* o *text*.

⇨ Dentro del elemento **xsl:template** debemos incluir la **plantilla** del **archivo de salida**, en el formato adecuado. Dentro de esta **plantilla** se pueden añadir **elementos de XSLT** que hagan referencia a nodos del archivo XML original.

3.3. Elementos de XSLT

Los elementos de XSLT más comunes son los siguientes:

<xsl:value-of select="*nodo*" />	Obtiene el **valor de un nodo** de XML (puede ser elemento, atributo…), indicado en el atributo *select* mediante una expresión de **XPath**.
<xsl:for-each select="*nodo*">	**Recorre** un conjunto de nodos indicados en el atributo *select* mediante una expresión de XPath, **aplicando una acción** a todos ellos.
<xsl:sort select="*nodo*" order=" " data-type=" "/>	**Ordena** un conjunto de nodos: <xsl:sort select="edad" order="ascending" data-type="number"/>
<xsl: text/>	**sirve para escribir texto literal en la salida** **<xsl:text>
</xsl:text> —> es el código ASCII/Unicode del salto de línea**
<xsl:if test="position() != last()">,</xsl:if>	**Esa línea se usa para controlar las comas en listas (sobre todo en JSON o CSV) y evitar poner una coma al final. SI NO ESTOY EN EL ÚLTIMO ELEMENTO, ESCRIBE UNA COMA.**

3.3. Elementos de XSLT

Ejemplo_7_8

Cuando estemos realizando la plantilla de un documento XSLT, a veces nos veremos obligados a introducir una **etiqueta dentro de otra**, en los casos en los que transformemos a XML o HTML. Esto se da cuando necesitamos introducir un valor del XML en forma de atributo. Por ejemplo, para una imagen:

```
<imagen>https://ejemplo.png</imagen>
```

XML

```
<img src="<xsl:value-of select="imagen"/>" >
```

HTML

Esto no se puede hacer, ya que no cumple con las reglas de sintaxis. Para ello, hay una etiqueta que nos permite definir atributos dentro de un elemento:

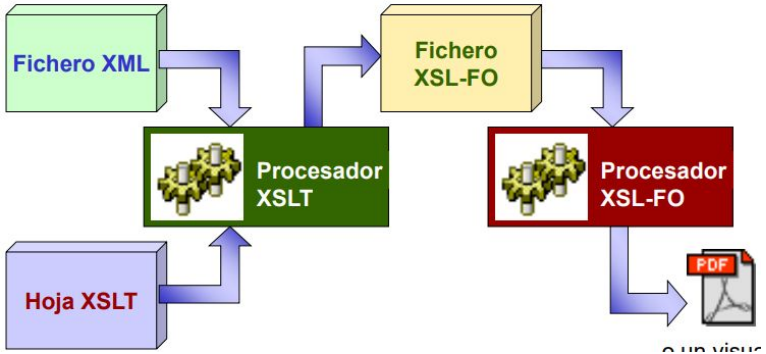
```
<xsl:attribute name="nombre-atributo">
```

La forma correcta sería:

```
<img>
    <xsl:attribute name="src">
        <xsl:value-of select="imagen"/>
    </xsl:attribute>
<img>
```

4. XSL-FO

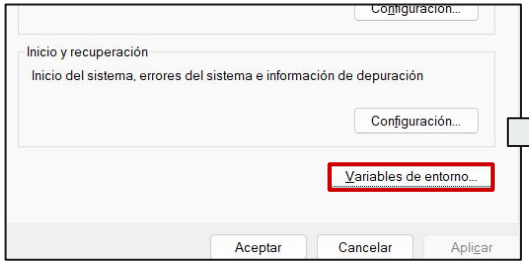
XSL-FO es el último lenguaje de marcas basado de la familia de XSL que nos queda por estudiar. Si XSLT se usa para **transformar** un archivo XML en otro distinto (ya sea XML, HTML o txt), este lenguaje permite describir **cómo se muestran los datos** en un **formato** determinado, definiendo aspectos como dimensiones, márgenes, párrafos, tablas, etc.
Gracias a **XSL-FO**, se pueden crear documentos en diversos **formatos** con un **estilo determinado**, como **PDF** (*Portable Document Format*), **PS** (*PostScript*), **RTF** (*Rich Text Format*), o incluso **PNG** (*Portable Network Graphics*).

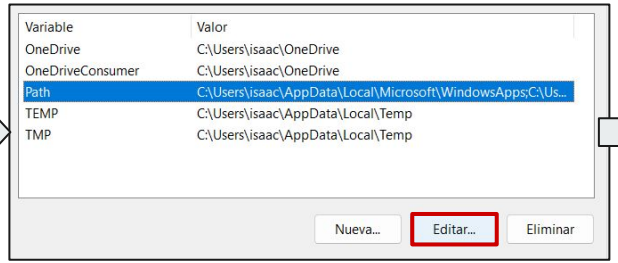


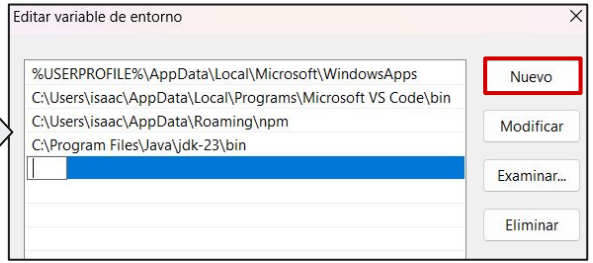
En este caso, también necesitaremos un **procesador** de XSL-FO. El más utilizado es **Apache FOP**, que se trabaja con comandos.

4.1. Instalación de Apache FOP

1. Descarga el archivo **binario** de Apache FOP desde el siguiente enlace: [Apache FOP]
2. **Descomprime** la carpeta, guárdala donde quieras y **copia la ruta** de la carpeta **fop** dentro de este directorio.
3. Vamos a **añadir esta ruta al PATH** para poder ejecutar los comandos. Para ello, accedemos a ***Panel de Control*** → ***Sistema y Seguridad*** → ***Sistema*** → ***Configuración avanzada del sistema*** (también podemos buscar directamente ***"Editar las variables de entorno del sistema"***). Dentro de ***Opciones avanzadas***, hacemos click en ***Variables de entorno***, seleccionamos ***Path*** y pulsamos ***Editar***. Finalmente, hacemos click en ***Nuevo*** y **pegamos la ruta** copiada anteriormente. Aceptamos todo para guardar.
4. Abrir una terminal y ejecutar el comando: ***fop -version*** para comprobar que todo ha ido bien.

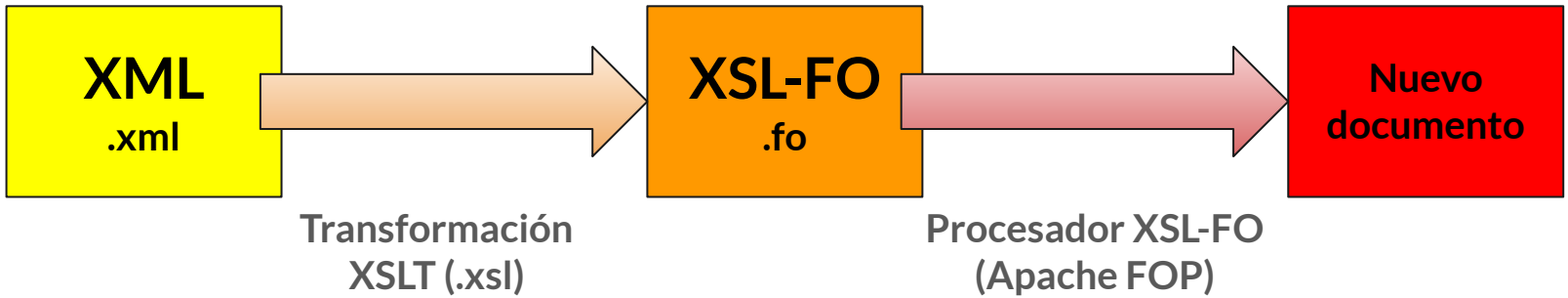






4.2. Estructura

Para ejecutar **XSL-FO** es necesario un archivo con extensión **.fo**, que es el que define cómo se va a ver el contenido. Este archivo, en lenguaje **XSL-FO**, es derivado de XML y con unas características y elementos propios, como los que hemos visto hasta ahora.
Sin embargo, el archivo **.fo** debe incluir, además de los elementos que indican el **formato** y **estilo**, los **contenidos** de este, que son parte del **XML original**. Por tanto, lo que se suele hacer es pasar primero el XML por una **transformación XSLT** para obtener el archivo **XSL-FO** que nos va a servir para realizar la transformación.



4.2. Estructura

El archivo XSL intermedio que nos sirve para obtener el archivo XSL-FO debe tener, por tanto, una estructura similar a la siguiente:

```
<?xml version="1.0" encoding="UTF-8" ?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
        <xsl:output method="xml" />
        <xsl:template match="/">
                <fo:root xmlns:fo="http://www.w3.org/1999/XSL/Format">
                        <fo:layout-master-set>
                                ...
                        </fo:layout-master-set>
                <fo:page-sequence master-reference="...">
                        ...
                </fo:page-sequence>
        </xsl:template>
<xsl:stylesheet>
```

Todo el contenido dentro de **xsl:template** se corresponde con el formato de archivo **XSL-FO**

4.3. Sintaxis

Por su lado, dentro de las características de XSL-FO, destaca por tener una **sintaxis** algo más compleja y precisa. En XSL-FO nos encontramos con los siguientes elementos:

fo:root → El elemento raíz de todo archivo XSL-FO. Debe contener un elemento **fo:layout-master-set**, y uno o más elementos **fo:page-sequence**

fo:layout-master-set → Define los estilos generales del formato de salida. Debe contener uno o más elementos **fo:simple-page-master** (uno por cada estilo de página que queramos)

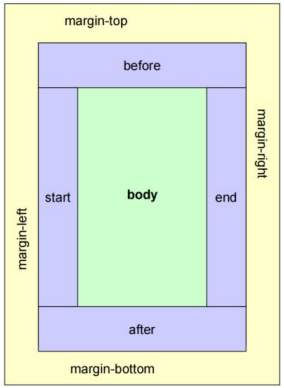
fo:simple-page-master → Contiene una **configuración de estilos para una página**, identificada con su *master-name*. Esta configuración se expresa mediante atributos y elementos:

Atributos

master-name: nombre de la configuración
page-width: ancho de la página
page-height: altura de la página
margin-top/margin-bottom/margin-left/ margin-right: márgenes en los cuatro costados por separado
margin: margen en todos los lados igual

Elementos

region-body: cuerpo de la página. Puede contener atributos de margen.
region-before: cabecera de la página
region-after: pie de página
region-start: lateral izquierdo
region-end: lateral derecho
Estos cuatro últimos elementos son opcionales, y deben tener el atributo *extent* para indicar su tamaño.



4.3. Sintaxis

Ejemplo_7_8

fo:page-sequence

Representa un contenedor de objetos de página (aunque si es necesario por tamaño, ocupará varias páginas). Contiene un **atributo** *master-reference* que equivale a uno de los *master-name* definidos en **fo:simple-page-master**, para determinar qué formato de página se va a usar. Como elementos, puede contener (en el mismo orden):

fo:static-content

Es opcional, y puede haber más de uno. **Agrupa contenidos** que estarán **fijos** en la página (por ejemplo, encabezados y pies de página). Deben tener el atributo ***flow-name*** con los valores **xsl-region-before**, **xsl-region-after**, **xsl-region-start** o **xsl-region-end,** indicando en cuál de estas regiones se sitúa el contenido.

fo:flow

Es obligatorio y solo puede haber uno. **Agrupa contenidos que "fluyen"**, es decir, que se van colocando uno tras otro, y no en una posición fija. Debe tener el atributo ***flow-name*** con el valor **xsl-region-body**, ya que son los contenidos de la región principal o cuerpo de la página.

A su vez, tanto en **fo:flow** como en **fo:static-content**, se admite como elemento **fo:block**:

fo:block

Agrupa contenido en un bloque (similar a un *div* en HTML). Puede contener cualquier texto u otros elementos para determinar títulos, encabezados, listas o tablas (no entraremos a definirlos). Además, se pueden definir atributos de estilos como ***font-size***, ***text-align***, ***color***, ***background-color***, ***padding***, ***font-family*** y muchos otros más (similares a CSS).

Más información sobre los atributos y elementos de fo:block

4.4. Conversión

Ejemplo_7_9

Una vez que tenemos el archivo de **XSL-FO**, obtenido a partir de una transformación XSLT para poder usar los datos de un XML, tan solo nos queda realizar la transformación mediante el **procesador de XSL-FO**, en nuestro caso, **Apache FOP**.
Para ello, abrimos una terminal en la misma carpeta que se ha generado nuestro archivo **.fo**, y ejecutamos el siguiente comando:

```
fop  archivo.fo  -tipo_de_formato  nombre_fichero_salida
```

Archivo **.fo** generado anteriormente

Formato al cual queremos convertir el documento (**pdf**, **rtf**, **ps**, **png**...)

Nombre que le queremos dar al archivo ya transformado, con su extensión correspondiente

Ejemplos

```
fop  ejemplo.fo -pdf ejemplo.pdf
```

```
fop  ejemplo.fo -ps ejemplo.ps
```

```
fop  ejemplo.fo -rtf ejemplo.rtf
```

```
fop  ejemplo.fo -png ejemplo.png
```